Motor de Tráfico Aéreo

Nodos AVL — Árbol Autobalanceado

Aeropuerto Internacional de Santa Ana

Materia: Estructuras de Datos / Programación II

Lenguaje: Rust

Estructura: Árbol AVL (Adelson-Velsky y Landis)

Clave de ordenamiento: Altitud en pies

FASE 1: Análisis de Seguridad y Propiedad

1.1 Documentación de Memoria: Option::take() y Ownership

En Rust, cada valor en memoria tiene exactamente UN dueño (ownership). Cuando un nodo del árbol AVL posee a sus hijos mediante Option<Box<Nodo>>, no podemos simplemente "copiar" un hijo para moverlo a otro lugar, ya que eso violaría la regla de un solo dueño.

El método Option::take() resuelve esto de forma segura en dos pasos:

- EXTRAE el valor del Option, transfiriendo el ownership al llamador.
- DEJA un None en la posición original, evitando referencias colgantes (dangling references).

Ejemplo en rotar_derecha():

```
let mut x = y.izquierdo.take().unwrap();
```

- → y.izquierdo ahora es None (ya no es dueño de x)
- → x es el nuevo dueño del subárbol izquierdo

```
y.izquierdo = x.derecho.take();
```

- → x.derecho pasa a ser None
- → y.izquierdo toma ownership del antiguo hijo derecho de x

```
x.derecho = Some(y);
```

- → x toma ownership de y como su nuevo hijo derecho

Sin take(), Rust no compilaría el código porque detectaría que estamos intentando usar un valor que ya fue movido (error E0382: use of moved value). Gracias a take(), cada transferencia de ownership es explícita y segura.

1.2 Concepto de Box<Nodo>

En Rust, el tamaño de cada tipo debe ser conocido en tiempo de compilación. Si definiéramos:

```
struct Nodo { izquierdo: Option<Nodo>, ... }
```

Tendríamos un tipo recursivo de tamaño INFINITO: un Nodo contiene otro Nodo, que contiene otro Nodo, ad infinitum. El compilador no puede reservar memoria para algo de tamaño desconocido.

Box<T> es un puntero inteligente que soluciona esto:

- Almacena el dato T en el HEAP (memoria dinámica).
- El Box en sí ocupa solo 8 bytes (un puntero) en el STACK.

- Esto da un tamaño FINITO y conocido en compilación.
- Cuando el Box se destruye (sale de ámbito), libera automáticamente la memoria del heap (patrón RAII), sin necesidad de free() manual.

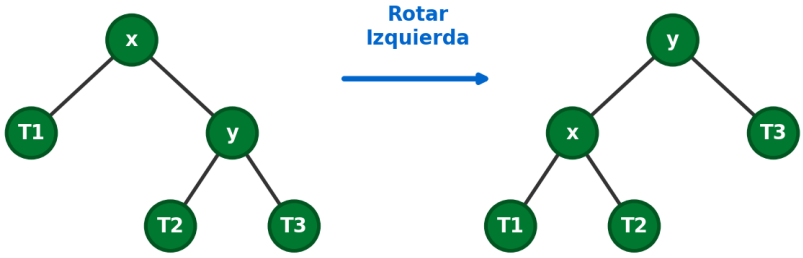
Así, Option<Box<Nodo>> ocupa exactamente 8 bytes (un puntero al heap) cuando contiene un valor, o se representa como un puntero nulo cuando es None, permitiendo la recursión de forma segura y eficiente.

1.3 Diagramas de Rotaciones AVL

Las rotaciones son las operaciones clave que mantienen el árbol AVL balanceado. A continuación se muestran los dos tipos de rotaciones simples:

Rotación Simple a la Derecha (Caso Left-Left):

Rotación Simple a la Izquierda (Caso Right-Right):



1.4 Prueba de Escritorio — Árbol AVL

Secuencia de inserciones: **[5000, 3000, 2000, 4000, 3500, 6000]**

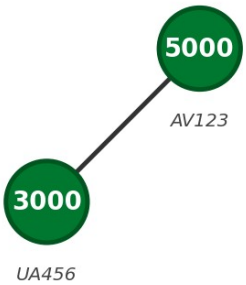
Paso 1: Insertar 5000 (AV123)

Árbol vacío → se crea el nodo raíz.

Balance de 5000: 0 → Sin rotación ✓

Paso 2: Insertar 3000 (UA456)

3000 < 5000 → va a la izquierda.

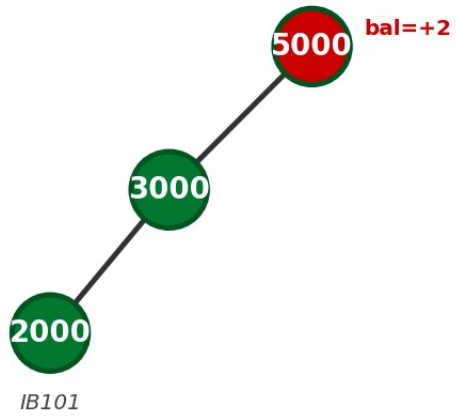


Balance de 5000: +1 → Sin rotación ✓

Paso 3: Insertar 2000 (IB101) — ROTACIÓN

2000 < 5000 → izquierda. 2000 < 3000 → izquierda.

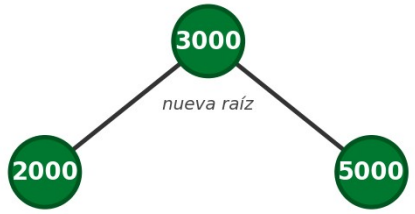
ANTES de rotar:



Diagnóstico: balance(5000) = 2 > 1. La inserción fue a la izquierda del hijo izquierdo (2000 < 3000).

Caso: Izquierda-Izquierda → ROTACIÓN SIMPLE A LA DERECHA sobre 5000

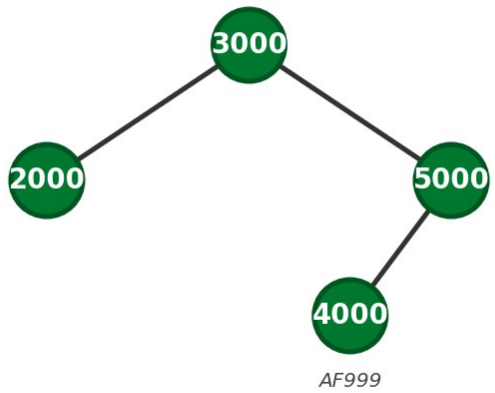
DESPUÉS de rotar:



Todos los balances = 0 ✓

Paso 4: Insertar 4000 (AF999)

4000 > 3000 → derecha. 4000 < 5000 → izquierda.

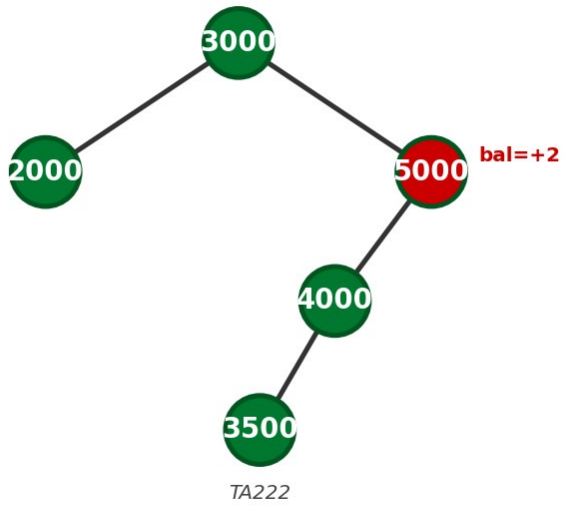


Balance de 5000: +1. Balance de 3000: -1 → Sin rotación ✓

Paso 5: Insertar 3500 (TA222) — ROTACIÓN

3500 > 3000 → der. 3500 < 5000 → izq. 3500 < 4000 → izq.

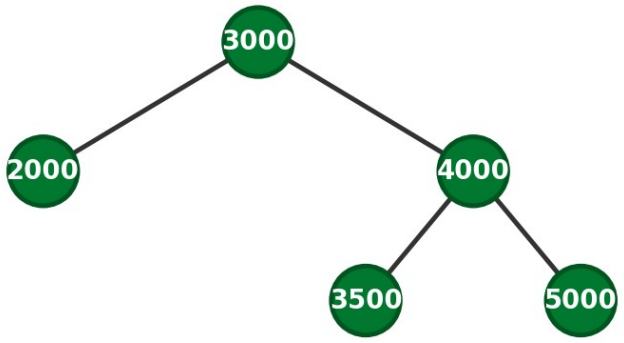
ANTES de rotar:



Diagnóstico: balance(5000) = 2 > 1. La inserción fue a la izquierda del hijo izquierdo (3500 < 4000).

Caso: Izquierda-Izquierda → ROTACIÓN SIMPLE A LA DERECHA sobre 5000

DESPUÉS de rotar:

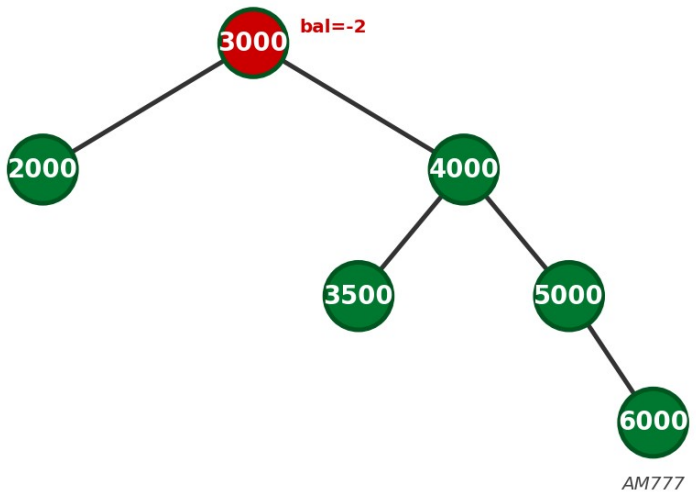


Balance de 4000: 0. Balance de 3000: -1 → OK ✓

Paso 6: Insertar 6000 (AM777) — ROTACIÓN

6000 > 3000 → der. 6000 > 4000 → der. 6000 > 5000 → der.

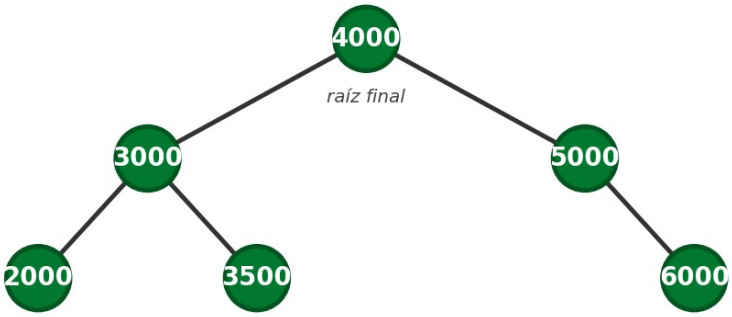
ANTES de rotar:



Diagnóstico: balance(3000) = -2 < -1. La inserción fue a la derecha del hijo derecho (6000 > 4000).

Caso: Derecha-Derecha → ROTACIÓN SIMPLE A LA IZQUIERDA sobre 3000

DESPUÉS de rotar — ÁRBOL FINAL:



Verificación de balances:

- Balance de 4000 (raíz): 2 - 2 = 0 ✓
- Balance de 3000: 1 - 1 = 0 ✓
- Balance de 5000: 0 - 1 = -1 ✓
- Hojas (2000, 3500, 6000): 0 ✓

→ Árbol perfectamente balanceado. Propiedad AVL cumplida.

Resumen de Rotaciones

Paso	Altitud Insertada	Nodo Desbalanceado	Balance	Tipo de Rotación
3	2000	5000	+2	Simple Derecha (L-L)
5	3500	5000	+2	Simple Derecha (L-L)
6	6000	3000	-2	Simple Izquierda (R-R)

Solo ocurrieron Rotaciones Simples (3 en total). No se presentaron casos de Rotación Doble (Izquierda-Derecha o Derecha-Izquierda).

Verificación In-Order: 2000 → 3000 → 3500 → 4000 → 5000 → 6000 (orden ascendente correcto ✓)

FASE 2: Localización de Vuelos

Se implementó la función buscar_vuelo() que localiza un vuelo por su altitud utilizando búsqueda binaria sobre el árbol AVL.

Firma:

```
fn buscar_vuelo<'a>(nodo: &'a Option<Box<Nodo>>, altitud: u32) -> Option<&'a Vuelo>
```

Características clave:

- Solo lectura: recibe referencias inmutables (&), impidiendo que el radar modifique datos accidentalmente.
- Eficiencia O(log n): en cada paso descarta la mitad del árbol, aprovechando la propiedad BST.
- Sin copias: retorna Option<&Vuelo> (una referencia al vuelo), no una copia del objeto.
- El lifetime 'a garantiza que la referencia retornada vive tanto como el árbol, evitando referencias colgantes.

FASE 3: Descenso y Aterrizaje (Eliminación)

Se implementó la función eliminar_vuelo() que remueve un avión del árbol cuando aterriza, manteniendo el balance AVL.

```
fn eliminar_vuelo(nodo_opt: Option<Box<Nodo>>, altitud: u32) -> Option<Box<Nodo>>
```

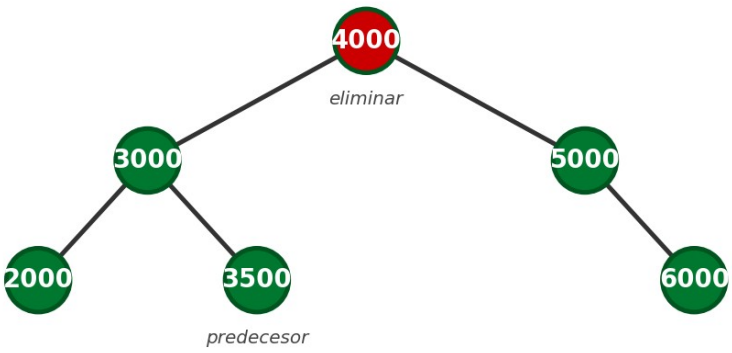
Los 3 casos de eliminación:

- Caso 1 — Nodo hoja (sin hijos): Se elimina directamente retornando None. El Box<Nodo> se destruye y Rust libera la memoria del heap automáticamente.
- Caso 2 — Un solo hijo: El nodo se reemplaza por su único hijo. take() transfiere el ownership del hijo y el nodo actual se destruye.
- Caso 3 — Dos hijos: Se busca el predecesor in-order (el mayor del subárbol izquierdo) usando obtener_maximo(). Se reemplazan los datos del nodo actual con los del predecesor. Luego se elimina el predecesor original del subárbol izquierdo recursivamente.

Tras la eliminación, se recalculan las alturas con actualizar_altura() y se ejecutan las rotaciones necesarias (usando obtener_balance() del hijo) para mantener la propiedad AVL (|balance| ≤ 1).

Ejemplo: Eliminar 4000 (Caso 3 — Dos hijos)

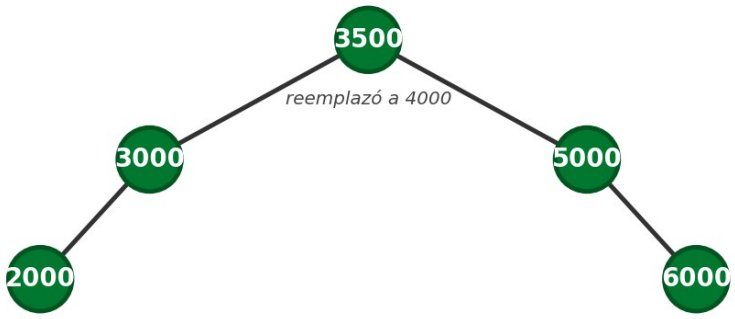
ANTES:



Predecesor in-order de 4000 = 3500 (el mayor del subárbol izquierdo).

Se reemplaza 4000 por 3500 y se elimina el nodo 3500 original.

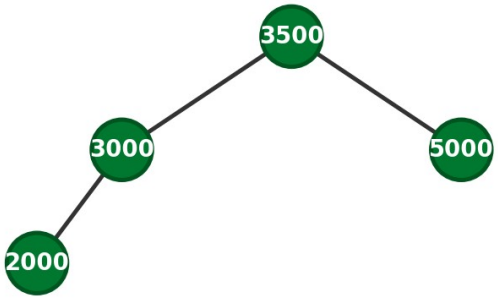
DESPUÉS:



El árbol permanece balanceado tras la eliminación. ✓

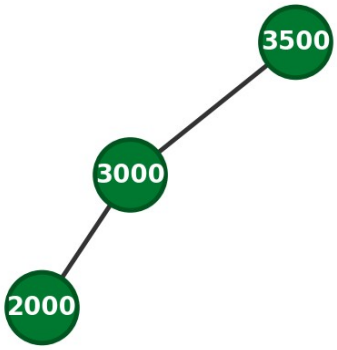
Eliminar 6000 (Caso 1 — Nodo hoja)

El nodo 6000 no tiene hijos → se elimina directamente.



Eliminar 5000 (Caso 1 — Nodo hoja)

El nodo 5000 ya no tiene hijos (6000 fue eliminado) → se elimina directamente.


3500
3000
2000

FASE 4: Alerta de Colisión (Opción A — Proximidad)

Se implementó la función vuelos_en_rango() que cuenta cuántos aviones están volando en un rango de altitud peligrosamente cercano.

```
fn vuelos_en_rango(nodo: &Option<Box<Nodo>>, min: u32, max: u32) -> usize
```

Eficiencia O(log n + k):

- Si la altitud del nodo < min: solo busca a la derecha (poda subárbol izquierdo).
- Si la altitud del nodo > max: solo busca a la izquierda (poda subárbol derecho).
- Si está dentro del rango: cuenta 1 y busca en ambos subárboles.

Ejemplo: vuelos_en_rango(&radar, 3000, 5000) = 4 vuelos

- (3000, 3500, 4000, 5000 están dentro del rango)